

- Intel 8051 Micro-controller family, its architecture and instruction sets
- Assembly language programming
- Interfacing with seven segment display

9.1 Intel 8051 Micro-controller family, its architecture and instruction sets

A. Introduction

Microcontroller is a small computer on a single IC which contains processor core along with memory, I/O ports and other features. Microcontrollers are used in embedded applications in which systems are controlled automatically to carry out certain application. Almost every system using microcontroller performs control-oriented tasks. Several peripheral devices are inbuilt within the microcontroller to carry out the specified function. Timers, ADC and serial communication devices are few examples of peripheral devices.

B. Block Diagram

The general block diagram of the microcontroller is shown in the figure below:

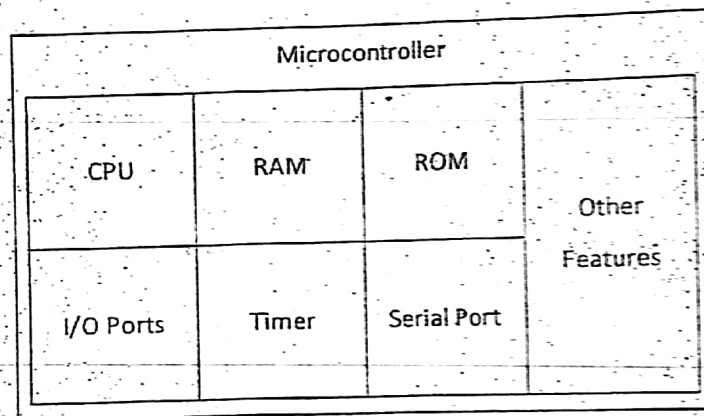


Figure 9.1: General block diagram of microcontroller

C. Comparison with Microprocessor

Many would easily presume that microcontroller and microprocessor to be similar. However, the following table will make a clear distinction between microcontroller and microprocessor.

SN	Microprocessor	Microcontroller
1.	General purpose processors	Special purpose processors.
2.	It contains complete functional CPU only	In addition to functional CPU, it has timers, I/O ports, internal RAM and ROM, and other features
3.	Designer can select the size of memory, number of I/O ports, timers etc to be used	Size of memory, number of I/O ports, timers etc are fixed for a particular microcontroller

4.	Clock speed in very high in GHz range	Clock speed is low in MHz range
5.	Powerful addressing modes and many instructions are available to move data between memory and CPU	It focuses on bit handling instructions along with byte processing instructions.
6.	Access time for external memory and I/O devices is more	Access time for on-chip memory and I/O devices is less
7.	Microprocessor based systems are expensive and consumes more power	Microcontroller based systems are cheap and consumes less power

D. Criteria For Choosing a Microcontroller

- It must meet the computational needs of the task efficiently and cost effectively. Other considerations includes:
 - Speed, packaging (DIP(dual line package), QFP(quad flat package)), power consumption, amount of RAM and ROM, number of I/O pins and the timer on the chip
 - Ease to amendments, cost per units
- It must provide flexibility to develop products around it. Some of the considerations include availability of an assembler, debugger, C compiler, emulator, technical support.
- It along with other reliable resources must be readily available in required quantities at any instant of time.

E. Comparison of 8051 Family Members ✓

Each member of 8051, somehow, differs from each other. Though the instruction sets are almost common, the features provided can vary. The 8031 microcontroller is also referred as ROMLESS 8051 as all features are common except ROM space. The following table shows comparison of 8051, 8052 and 8031 microcontrollers.

Table 9.1: Comparison of three microcontrollers

Feature	8051	8052	8031
ROM	4K	8K	0K
RAM (bytes)	128	256	128
Timers	2	3	2
I/O Pins	32	32	32

Serial Port	1	1	1
Interrupt Sources	6	8	6

F. 8051 ARCHITECTURE

Internal Block Diagram of 8051

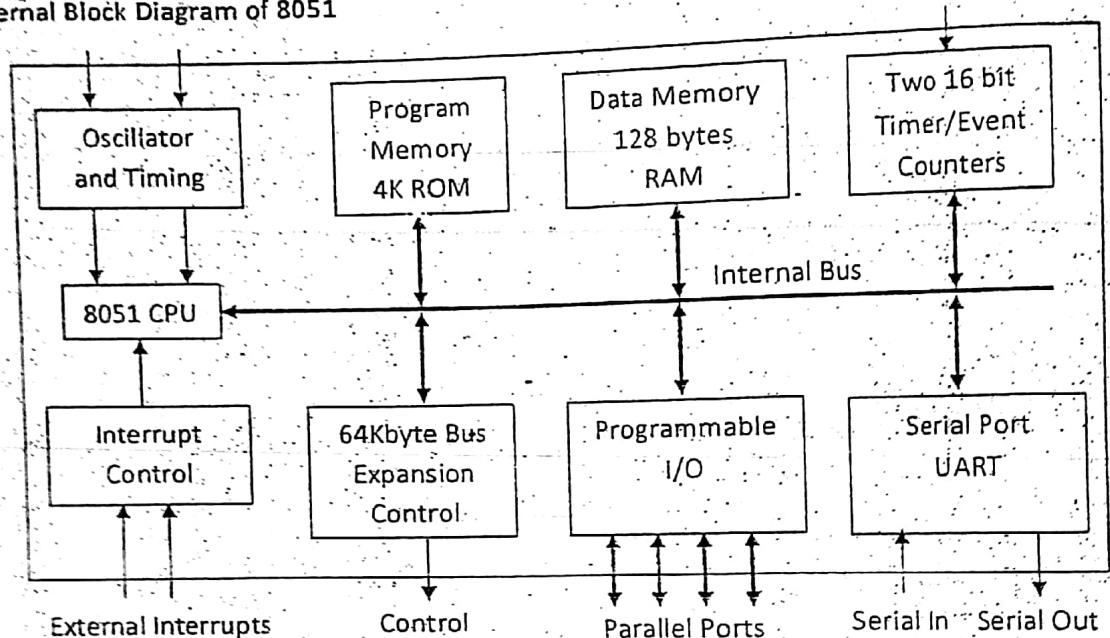


Figure 9.2: Internal block diagram of 8051 Architecture

Features of 8051 Architecture

- **Eight bit CPU with registers A and B:** Register A or Accumulator is used for mathematical and data transfer operations. Register B is used for multiplication and division purpose.
- **Sixteen bit program counter (PC) and data pointer (DPTR):** PC points to the address of next instruction to be executed from ROM while DPTR is used to point to the memory addresses for internal and external code access and external data access. DPTR is made up of two 8 bit registers, DPH and DPL.
- **Eight bit program status word (PSW):** Four flags in PSW are used to represent the outcomes of mathematical operations; Carry (CY), Auxiliary Carry (AC), Overflow (OV), and Parity (P) flags. It also consists two register select bits which select the particular register bank; RS1 and RS0 determines which register bank is being used out of four register banks.
- **Eight bit stack pointer (SP):** SP points to the stack which is the area to store and retrieve data quickly for some operations. It follows last in first out technique.

- **Internal ROM:** It consists of 4 Kbytes or memory space as program memory. Look up tables can also be stored which can be accessed using appropriate instruction.
- **Internal RAM:** It consists of 128 bytes of memory space as data memory.
 - Four Register Banks, each with eight registers (R0 – R7): Bank 0 occupies address from 00H to 07H and consecutive addresses are used by bank 1, bank 2 and bank 3. Total 32 registers are available from address 00H to 1FH. Bank 0 is selected as default. RS1 = 0 and RS0 = 1 in PSW register will select the register bank 1.
 - Sixteen bytes of bit addressable memory: The address from 20H to 2FH of RAM is bit addressable. It is useful in bit manipulating operations. Each bit can be addressed using direct address from 00H to 7FH.
 - Eighty bytes of general purpose data memory: The memory space from address 30H to 7FH can be used for various operations when required.

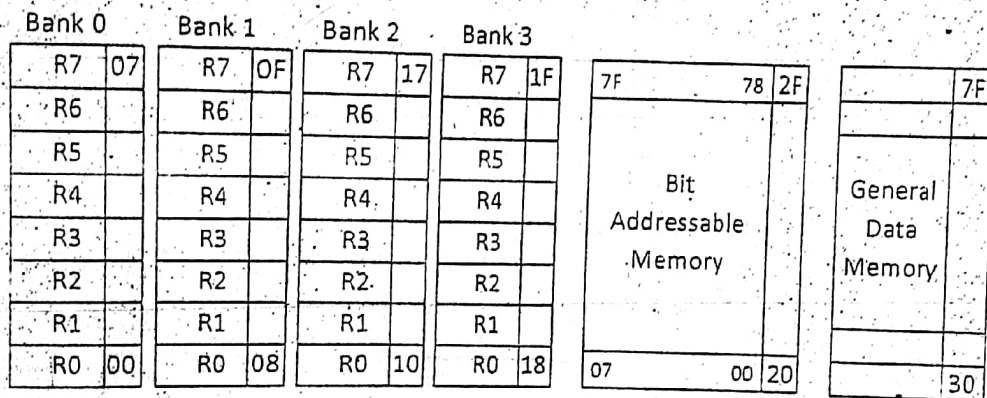


Figure 9.3: Internal RAM Organization

- Thirty two I/O pins arranged as four 8 bit ports: Four ports are bidirectional and can be used for input and output. Some of the pins are multifunctional which provide other functions along with input and output.
- Two 16 bit timer/counters (T0 and T1): Each counter can be programmed to count internal clock pulses, acting as a timer, or programmed to count external pulses as a counter. This selection as well as mode of operation of counter can be set by using Timer Mode Control (TMOD) register.
- Full Duplex serial data receiver/transmitter: Register Serial Control (SCON) controls serial data communication, and pins RXD and TXD are used to connect to other devices supporting serial

- communication. The Serial Buffer (SBUF) register is used to hold data in serial communication process.
- Two external interrupts and three internal interrupt sources: Interrupt Enable (IE) register selects which interrupt is to be selected and enabled. INTO and INT1 pins are used by external circuitry to interrupt processor. Timer overflow (TF), receive interrupt (RI), and transmit interrupt (TI) are internal interrupts.
 - The 8051 Oscillator and Clock: It is the circuitry that generates the clock pulses by which all internal operations are synchronized. Time for particular instruction execution can be calculated based on number of machine cycles required by the instruction. For AT89S51, operating frequency is 11.0592 and its machine cycle consists of 12 clocks. So, time period for one machine cycle is 12 times the time period of single pulse which is equivalent to $1.085\mu s$.

8051 Special Function Registers (SFRs)

They are a group of specific internal registers that use internal RAM and their address lie between 80H and FFH. Some SFRs are bit addressable which allows programmer to access each bit of the register. The list of SFRs is given in the table below:

Table 9.2: List of Special Function Registers

Name	Description	RAM Address	Access Level
A	Accumulator	0E0H	Bit Addressable
B	Register B, for multiplication and division	0F0H	Bit Addressable
PSW	Program Status Word	0D0H	Bit Addressable
SP	Stack Pointer	81H	
DPH	Data Pointer Higher Byte	83H	
DPL	Data Pointer Lower Byte	82H	
IE	Interrupt Enable	0A8H	Bit Addressable
IP	Interrupt Priority	0B8H	Bit Addressable
P0	Port 0	80H	Bit Addressable
P1	Port 1	90H	Bit Addressable
P2	Port 2	0A0H	Bit Addressable
P3	Port 3	0B0H	Bit Addressable
PCON	Power Control	87H	

SCON	Serial Port Control	98H	Bit Addressable
SBUF	Serial Port Data Buffer	99H	
TMOD	Timer/Counter Mode Control	89H	
TCON	Timer/Counter Control	88H	Bit Addressable
TLO	Timer 0 Low Byte	8AH	
TH0	Timer 0 High Byte	8CH	
TL1	Timer 1 Low Byte	8BH	
TH1	Timer 1 High Byte	8DH	

G. Pin Descriptions

Pins 1 – 8 (PORT-1)

These eight pins represent PORT 1 which can be used as an input/output port. Since it is internally pulled up, it can be used without any external pull up registers configuration.

Pin – 9 (RESET)

RESET pin is used to set different registers to its initial values. The RESET pin must be set high for 2 machine cycles.

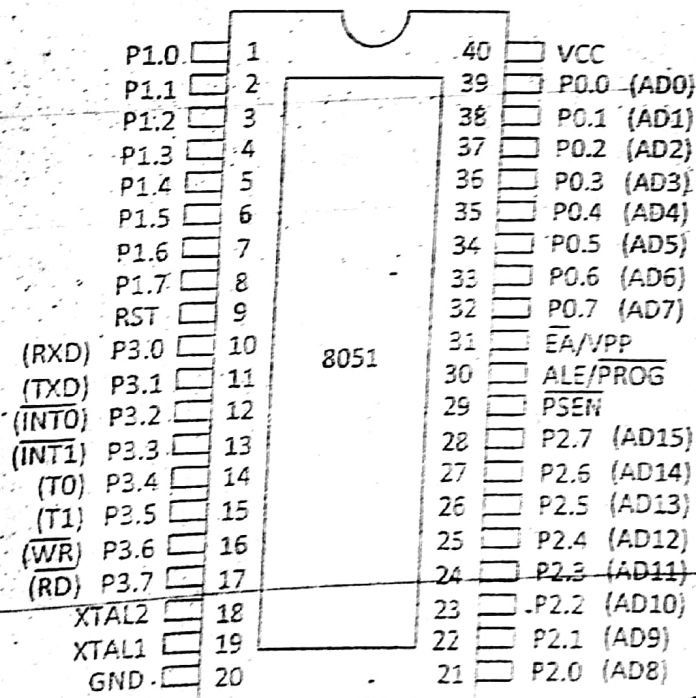


Figure 9.4: Pin descriptions of 8051 microcontroller

Pins 10 – 17 (PORT 3)

These pins together called PORT 3 are bi directional and multifunctional in nature. Similar to port 1, it can be used as input or output without any external pull up registers configuration. Besides I/O, it supports serial communication (RXD and TXD), external interrupts (INT0 and INT1), timers (T0 and T1), and control signals (WR and RD) for external memory.

Pins 18 – 19 (XTAL)

These pins are used to connect an external crystal to provide system clock.

Pin – 20 (GND, 0V)

Pins 21 – 28 (PORT 2)

These pins are bidirectional and multifunctional in nature. PORT 2 may be used as an input or output port similar to port 1. The alternate use of port 2 is to provide a high-order address in conjunction with the port 0 to address external memory.

Pin 29 (PSEN)

Program Store Enable (PSEN) is connected to Output Enable (OE) pin of external memory being interfaced. It is an active low output signal. When this pin is reset, microcontroller can read content of external memory location.

Pin 30 (ALE)

Address Latch Enable (ALE) is used to select address or data signal that are required while interfacing external memory. It is active high output signal and when it goes high, the lower address provided by port 0 is latched into the external address latch. This pin is also the program pulse input during flash programming.

Pin 31 (EA)

External Access enables or disables access of program from external memory. It must be connected to GND to fetch code from external program memory locations. It should be strapped to VCC for program executions of internal memory.

Pins 32 - 39 (PORT 0)

PORT 0 is a collection of open drain bidirectional I/O pins. It can be configured as low-order address or data bus while accessing external memory.

Pin 40 (VCC, +5V)

H. Minimum Hardware Configuration

Power Supply: Pin 40 is connected to +5VDC, Pin 20 is grounded. Pin 31 is connected to VCC, representing the code is accessed from internal memory.

Reset Circuit: Charging of capacitor makes RST high, which ensures two machine cycles on RST pin. After completion of charge, capacitor blocks DC causing RST low.

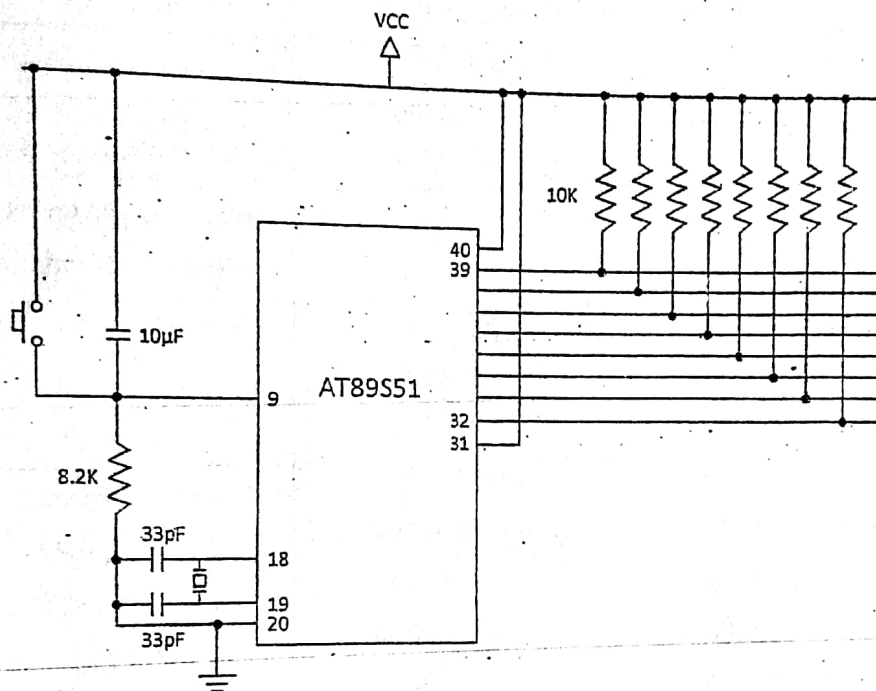


Figure 9.5: Minimum configuration for microcontroller to operate

Oscillator circuit: Ceramic capacitors of value between $20\mu\text{F}$ – $40\mu\text{F}$ are used as stabilizing capacitors. They act as loading capacitor and adjust the crystal frequency by shifting the frequency to a lower value.

Pull up Circuit: Pins of PORT 0 are open drain, so require pull up circuit. Each pin must be connected externally to a 10K ohm pull-up resistor.

I. 8051 Instruction Sets

Different symbols are used in the instruction whose meaning is clarified below:

#data – represents 8 bit data

Rn – represents one of eight registers (R0, R1, R2, R3, R4, R5, R6, R7)

@Ri – represents address pointed by value of Ri. Ri can be either R0 or R1.

direct – represents direct byte addressable memory

bit – direct bit addressable memory

C – Carry, A – Accumulator, B – Register B

addr11 – 11 bit address, addr16 – 16 bit address and rel – 8 bit relative address

1. Data Transfer Instructions

SN	Mnemonics	Operation	Description
1	MOV A, Rn	$A \leftarrow Rn$	MOV instruction is used to transfer data involving registers, memory and immediate data
2	MOV A, direct	$A \leftarrow [\text{direct}]$	
3	MOV A, @Ri	$A \leftarrow [Ri]$	
4	MOV A, #data	$A \leftarrow \text{data}$	
5	MOV Rn, A	$Rn \leftarrow A$	
6	MOV Rn, direct	$Rn \leftarrow [\text{direct}]$	
7	MOV Rn, #data	$Rn \leftarrow \text{data}$	
8	MOV direct, A	$[\text{direct}] \leftarrow A$	MOV instruction is used to transfer data involving registers, memory and immediate data
9	MOV direct, Rn	$[\text{direct}] \leftarrow Rn$	
10	MOV direct, direct	$[\text{direct}] \leftarrow [\text{direct}]$	
11	MOV direct, @Ri	$[\text{direct}] \leftarrow [Ri]$	
12	MOV direct, #data	$[\text{direct}] \leftarrow \text{data}$	
13	MOV @Ri, A	$[Ri] \leftarrow A$	
14	MOV @Ri, direct	$[Ri] \leftarrow [\text{direct}]$	
15	MOV @Ri, #data	$[Ri] \leftarrow \text{data}$	MOV instruction is used to transfer data involving registers, memory and immediate data
16	MOV DPTR, #data16	$DPTR \leftarrow \text{data16}$	
17	MOVC A, @A + DPTR	$A \leftarrow [A + DPTR]$	
18	MOVC A, @A + PC	$A \leftarrow [A + PC]$	
19	MOVX A, @Ri	$A \leftarrow [Ri]$	
20	MOVX A, @DPTR	$A \leftarrow [DPTR]$	
21	MOVX @Ri, A	$[Ri] \leftarrow A$	
22	MOVX @DPTR, A	$[DPTR] \leftarrow A$	MOVX is used to move data to and from external RAM. R0, R1 and DPTR are used to hold address of RAM
23	PUSH direct	$\text{Stack} \leftarrow [\text{direct}]$	
24	POP direct	$[\text{direct}] \leftarrow \text{Stack}$	
25	XCH A, Rn	$A \leftrightarrow Rn, Rn \leftarrow A$	

26	XCH A, direct	$A \leftarrow [\text{direct}], [\text{direct}] \leftarrow A$	
27	XCH A, @Ri	$A \leftarrow [Ri], [Ri] \leftarrow A$	
28	XCHD A, @Ri		

2. Arithmetic Instructions

SN	Mnemonics	Operation	Description
1	ADD A, Rn	$A \leftarrow A + Rn$	Accumulator is one of the sources as well as destination for every ADD and SUB instructions
2	ADD A, direct	$A \leftarrow A + [\text{direct}]$	
3	ADD A, @Ri	$A \leftarrow A + [Ri]$	
4	ADD A, #data	$A \leftarrow A + \text{data}$	
5	ADDC A, Rn	$A \leftarrow A + Rn + C$	
6	ADDC A, direct	$A \leftarrow A + [\text{direct}] + C$	
7	ADDC A, @Ri	$A \leftarrow A + [Ri] + C$	
8	ADDC A, #data	$A \leftarrow A + \text{data} + C$	
9	SUBB A, Rn	$A \leftarrow A - Rn - C$	
10	SUBB A, direct	$A \leftarrow A - [\text{direct}] - C$	
11	SUBB A, @Ri	$A \leftarrow A - [Ri] - C$	
12	SUBB A, #data	$A \leftarrow A - \text{data} - C$	
13	INC A	$A \leftarrow A + 1$	
14	INC Rn	$Rn \leftarrow Rn + 1$	
15	INC direct	$[\text{direct}] \leftarrow [\text{direct}] + 1$	
16	INC @Ri	$[Ri] \leftarrow [Ri] + 1$	
17	DEC A	$A \leftarrow A - 1$	
18	DEC Rn	$Rn \leftarrow Rn - 1$	
19	DEC direct	$[\text{direct}] \leftarrow [\text{direct}] - 1$	
20	DEC @Ri	$[Ri] \leftarrow [Ri] - 1$	
21	INC DPTR	$DPTR \leftarrow DPTR + 1$	
22	MUL AB	$A \leftarrow \text{Lower Byte}$ $B \leftarrow \text{Higher byte}$	
23	DIV AB	$A \leftarrow \text{Quotient}$ $B \leftarrow \text{Remainder}$	
24	DAA	Decimal Adjust accumulator	

3. Logical Instructions

SN	Mnemonics	Operation	Description
1	ANL A, Rn	$A \leftarrow A \text{ AND } Rn$	Logical AND, OR and XOR allows direct operation on memory address as well.
2	ANL A, direct	$A \leftarrow A \text{ AND } [\text{direct}]$	
3	ANL A, @Ri	$A \leftarrow A \text{ AND } [Ri]$	
4	ANL A, #data	$A \leftarrow A \text{ AND } \text{data}$	
5	ANL direct, A	$[\text{direct}] \leftarrow [\text{direct}] \text{ AND } A$	
6	ANL direct, #data	$[\text{direct}] \leftarrow [\text{direct}] \text{ AND } \text{data}$	
7	ORL A, Rn	$A \leftarrow A \text{ OR } Rn$	
8	ORL A, direct	$A \leftarrow A \text{ OR } [\text{direct}]$	
9	ORL A, @Ri	$A \leftarrow A \text{ OR } [Ri]$	
10	ORL A, #data	$A \leftarrow A \text{ OR } \text{data}$	
11	ORL direct, A	$[\text{direct}] \leftarrow [\text{direct}] \text{ OR } A$	
12	ORL direct, #data	$[\text{direct}] \leftarrow [\text{direct}] \text{ OR } \text{data}$	
13	XRL A, Rn	$A \leftarrow A \text{ XOR } Rn$	
14	XRL A, direct	$A \leftarrow A \text{ XOR } [\text{direct}]$	
15	XRL A, @Ri	$A \leftarrow A \text{ XOR } [Ri]$	
16	XRL A, #data	$A \leftarrow A \text{ XOR } \text{data}$	
17	XRL direct, A	$[\text{direct}] \leftarrow [\text{direct}] \text{ XOR } A$	
18	XRL direct, #data	$[\text{direct}] \leftarrow [\text{direct}] \text{ XOR } \text{data}$	
19	CLR A	$A \leftarrow 0$	
20	CPL A	$A \leftarrow A'$	
21	RL A	Rotate A left	
22	RLC A	Rotate A left through C	
23	RR A	Rotate A right	
24	RRC A	Rotate A right through C	
25	SWAP A	Swap nibbles of A	

4. Bit Manipulation and Program Branching Instructions

SN	Mnemonics	Operation	Description
----	-----------	-----------	-------------

1	CLR C	$C \leftarrow 0$	
2	CLR bit	$\text{bit} \leftarrow 0$	
3	SETB C	$C \leftarrow 1$	
4	SETB bit	$\text{bit} \leftarrow 1$	
5	CPL C	$C \leftarrow C'$	
6	CPL bit	$\text{bit} \leftarrow \text{bit}'$	
7	ANL C, bit	$C \leftarrow C \text{ AND bit}$	
8	ANL C, /bit	$C \leftarrow C \text{ AND bit}'$	
9	ORL C, bit	$C \leftarrow C \text{ OR bit}$	
10	ORL C, /bit	$C \leftarrow C \text{ OR bit}'$	
11	MOV C, bit	$C \leftarrow \text{bit}$	
12	MOV bit, C	$\text{bit} \leftarrow C$	Short jumps must be within
13	JC rel	Jump if $C \leftarrow 1$	128 to +127 bytes of the
14	JNC rel	Jump if $C \leftarrow 0$	contents of PC
15	JB bit, rel	Jump if $\text{bit} \leftarrow 1$	
16	JNB bit, rel	Jump if $\text{bit} \leftarrow 0$	Long Jumps and calls can be
17	JBC bit, rel	Jump if $\text{bit} \leftarrow 1$, and $b \leftarrow 0$	used for any location within
18	ACALL addr11	Absolute jump to routine	64 Kbyte address space
19	LCALL addr16	Long jump to routine	
20	RET	Return from subroutine	Absolute jumps and calls can
21	RETI	Return from interrupt	be used for address within
22	AJUMP addr11	Absolute jump	2Kbyte range
23	LJUMP addr16	Long jump	
24	SJMP rel	Short jump	
25	JMP @ A + DPTR	Jump relative to DPTR	
26	JZ rel	Jump if A is zero	
27	JNZ rel	Jump if A is not zero	
28	CJNE A, direct, rel		
29	CJNE A, #data, rel	Compare and jump if not	
30	CJNE Rn, #data, rel	equal	
31	CJNE @Ri, #data, rel		

32	DJNZ Rn, rel	Decrease and jump if not	
33	DJNZ direct, rel	zero	

J. Addressing Modes In 8051

Immediate Addressing Mode

The source operand is a constant value which must be preceded by # sign. It is used to load direct values into registers. For example, MOV A, #25H will assign 25 to register A.

Register Direct Addressing Mode

The operand is a register which holds the data to be manipulated. For example, ADD A, R5 will add content of A and R5, and store back in A.

Register Indirect Addressing Mode

Register is used to point the effective address of the operand. Registers R0, R1 and DPTR are used as pointer registers which must be preceded by @ sign. For example, MOV A, @R0 represents copying the contents of the address in R0 to the accumulator.

Direct Addressing Mode

The operand represents the actual address of RAM in the instruction. For instance, MOV A, 80H moves the data of 80H into accumulator.

Relative Addressing

A relative address or offset is added to the PC to form the actual address. Generally used in jump instructions.

Absolute Addressing Mode

In instructions, 11-bit or 16-bit absolute address is specified as the operand. ACALL and AJMP instructions use 11-bit address while LCALL and LJP use 16-bit address.

Indexed Addressing Mode

Index value or displacement is added to the base address to generate the effective address of the operand. For instance, MOVC A, @A + DPTR uses indexed addressing mode. The content pointed by (A + DPTR) address of ROM is copied to accumulator.

9.2 Assembly language programming

An assembly language program consists of series of statements which include assembly language instructions and directives. Assembly language instruction represents the operation to be carried out by the processor. Every instruction is composed of mnemonic followed by one, two or no

operand. Mnemonic represents the actual operation to be done which operands are data items being manipulated. Directives are used to give directions to the assembler. Generally used directives are DB, ORG, END, and EQU. The DB directive is used to define 8-bit data. The ORG represents the beginning of the program address while END represents the end of program. The EQU directive is used to define constant within a program. The numbers used must be followed by H to represent hex value otherwise the value will be taken as decimal.

The assembly language program, in general, is written using following format:

[label:] Mnemonic [operands] [; comments]

Example 1: Read the content of port1 and port2, OR those contents and store the result in external RAM location 0310.

Problem Analysis: Port1 (with address 90H) and Port2 (with address A0H) provide eight bit data, so after OR operation the final result will also be of eight bit. Hence, single byte memory location is enough to store the result. However, to store the result in external RAM, the address of external RAM must be loaded into DPTR register and MOVX instruction should be used for data transfer.

Source Code:

```
ORG 00H
MOV A, 90H           ; copy the data of port1 to A
ORL A, 0A0H          ; OR the contents of A with port2, and stored in A
MOV DPTR, #0310H     ; DPTR used to point the external RAM address
MOVBX @DPTR, A       ; move the content of A to RAM location pointed by DPTR
END
```

Example 2: Read the content of internal RAM locations 27H and 28H, add them and store the result in RAM locations 30H and 31H.

Problem Analysis: The largest possible value at memory locations can be FFH, so the maximum value of final result will be $FFH + FFH = 01FEH$. Hence, two bytes of memory is required to store the result. To solve this, ADD instruction must be used to add two data while ADDC or JNC/JC can be used to add the carry.

Source Code:

```
ORG 00H
MOV 30H, #00H        ; [30] ← 00, assigns zero to memory location 30H
MOV A, 27H            ; A ← [27], assigns content of location 27H to accumulator
```

```

ADD A, 28H          ;  $A \leftarrow A + [28]$ , adds content of A and memory location 28H
MOV 31H, A          ;  $[31] \leftarrow A$ , moves the lower byte or result to 31H address of RAM
MOV A, #00H         ;  $A \leftarrow 00$ , reset accumulator
ADDC A, 30H         ;  $A \leftarrow A + [30] + C$ , to extract the value of carry
MOV 30H, A          ;  $[30] \leftarrow A$ , move upper byte to location 30.
END

```

Example 3: Add the content of internal RAM location 29H and port1, and store the result in RAM locations 30H and 31H in BCD form.

Problem Analysis: In BCD, the largest possible value can be 99, so the maximum value of final result will be $99 + 99 = 198$. Hence, two bytes of memory is required to store the result. To solve this, ADD instruction must be used to add two BCD data and DA instruction after addition. However, DA instruction is not required for upper byte of result as it is less than 10. But, had there been more numbers causing upper byte to exceed more than 9, DA would have been required for upper byte as well.

Source Code:

```

ORG 00H
MOV 30H, #00H       ;  $[30] \leftarrow 00$ , assigns zero to memory location 30H
MOV A, 90H          ;  $A \leftarrow [90]$ , assigns content of Port1 to accumulator
ADD A, 29H          ;  $A \leftarrow A + [29]$ , adds content of accumulator and location 29H
DA A                ; Adjust the content of accumulator to BCD form
MOV 31H, A          ;  $[31] \leftarrow A$ , moves the lower byte or result to 31H address of RAM
MOV A, #00H         ;  $A \leftarrow 00$ , reset accumulator
ADDC A, 30H         ;  $A \leftarrow A + [30] + C$ , to extract the value of carry
MOV 30H, A          ;  $[30] \leftarrow A$ , move upper byte to location 30.
END

```

Example 4: Add 10 bytes of data of RAM location starting from address 20H. Store lower byte at 30H and upper byte at 31H.

Problem Analysis: Use of direct RAM address can make program complex, so R0 or R1 can be used to point the one byte address of RAM location. The register (R0 or R1) then can be incremented to access data of consecutive locations. The final sum after addition of 10 bytes of data can result in

two bytes of data. So, carry must be checked after addition and value in register must be incremented accordingly when carry is resulted after addition.

Source Code:

```

ORG 00H
MOV R0, #20H      ; R0 ← 20H, assigning starting address of RAM to R0
MOV R5, #0AH      ; R5 ← 0AH, counter for 10 bytes of data
MOV R6, #00H      ; R6 ← 00H, used for lower byte result
MOV R7, #00H      ; R7 ← 00H, used for upper byte result

HOME:
MOV A, @R0        ; A ← [R0], move content of RAM location pointed by R0 to A
ADD A, R6          ; A ← A + R6, adds data of RAM location in each iteration
JNC NEXT          ; checking if carry is generated after addition
    INR R7         ; R7 ← R7 + 1, carry after each addition is added to form upper byte

NEXT:
MOV R6, A          ; R6 ← A, move partial sum to R6
INR R0             ; R0 ← R0 + 1, increment the address of RAM to access next byte
DJNZ R5, HOME      ; decrease R5 by 1 and jump to HOME if R5 is not equal to zero
MOV 30H, R6        ; [30] ← R6, move the lower byte to memory address 30H
MOV 31H, R7        ; [31] ← R7, move the upper byte to memory address 31H
END

```

Example 5: ASCII character string is stored in the program memory starting at 300H. Send each character to port 2.

Problem Analysis: Since the data is stored in the program memory (ROM), DPTR must be used to point the address of ROM. Using MOVC instruction, the data can be retrieved and further manipulated as required.

Source Code:

```

ORG 00H
MOV DPTR, #300H    ; load address of data into DPTR

HOME:
CLR A              ; A ← 0, clear the content of A
MOVC A, @A + DPTR  ; A ← [A + DPTR], load ROM content of address (A + DPTR) to A

```

```

JZ EXIT ; jump out of loop if last character is detected, which is 0
MOV P2, A ; P2 ← A, move content of accumulator to port 2
INC DPTR ; DPTR ← DPTR + 1, to point to next character of ROM
SJMP HOME ; Continue the loop

ORG 300H
DB "ASSEMBLY PROGRAM", 0
EXIT
NOP
END

```

9.3 Interfacing with seven segment display

A. Seven Segment Configurations

Pin Configurations

The figure shows the pin configuration of seven segment display which consists of 10 pins; eight pins to control the leds and two common pins which are grounded or connected to VCC based on common cathode or common anode configuration.

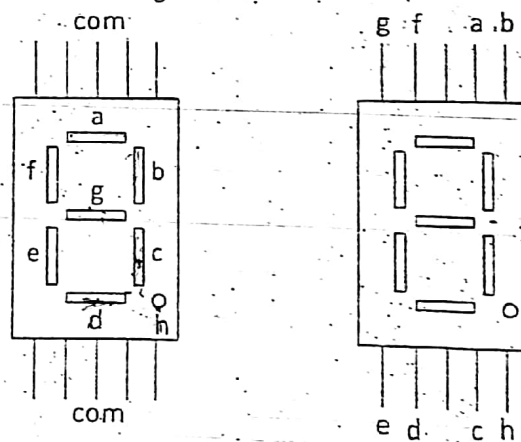


Figure 9.6: Pin Configurations of Seven Segment Display

Modes of Configurations

There are two modes of configurations: Common Anode Configuration and Common Cathode Configuration. In common anode configuration, anodes of all leds are connected together to form a common pin which must be connected to high logic voltage. In common cathode configuration, cathodes of all leds are connected together to form a common pin which must be connected to low logic voltage.

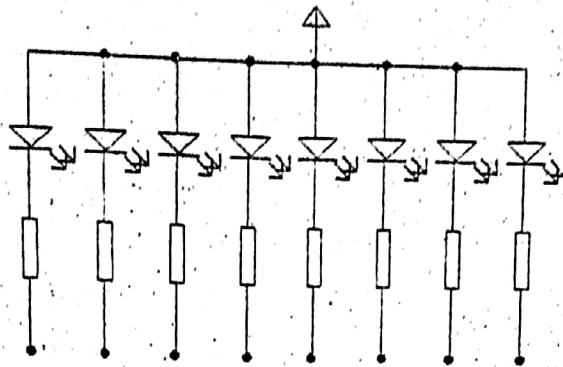


Figure 9.7: Common Anode Configurations

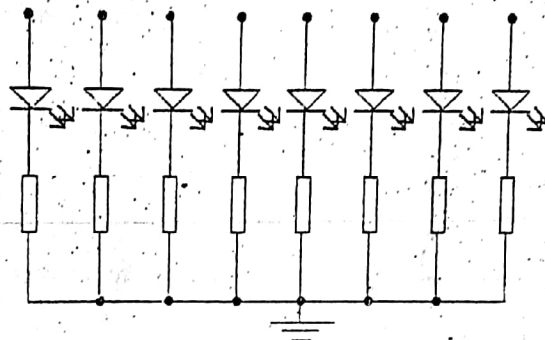


Figure 9.8: Common Cathode Configurations

Lookup table of HEX equivalent

For common anode configurations, low logic must be provided to the pins of the seven segment display to glow the particular led. The equivalent hex values are sent through the port of microcontroller. However, designer must be aware of the driving circuit which should provide the equivalent hex to the pins of seven segment display.

Common Anode Configurations									For Common Cathode mode
Digits	Individual LEDs Illuminated								HEX
	h	g	f	e	d	c	b	a	
0	1	1	0	0	0	0	0	0	0xC0
1	1	1	1	1	1	0	0	1	0xF9
2	1	0	1	0	0	1	0	0	0xA4
3	1	0	1	1	0	0	0	0	0xB0
4	1	0	0	1	1	0	0	1	0x99
5	1	0	0	1	0	0	1	0	0x92

6	1	0	0	0	0	0	1	0	0x82	0x7D
7	1	1	1	1	1	0	0	0	0xF8	0x07
8	1	0	0	0	0	0	0	0	0x80	0x7F
9	1	0	0	1	0	0	0	0	0x90	0x6F

B. Interfacing Seven Segment

Before connecting the seven segment display to the port of microcontroller, the current requirement of the seven segment display along with the source current and sink current capacity of the microcontroller must be examined. However, it is always better to use the driving circuit rather to connect seven segment display directly to the port of microcontroller.

Hardware Connections

Any port of the microcontroller can be used to connect to the seven segment display through the driving circuit. The driving circuit may be in the form of an IC which can sink or source high current. The circuit configurations can vary depending on the designer. IC ULN2003 is an example of a current sinker while IC L293D can be a good source of current for driving circuits.

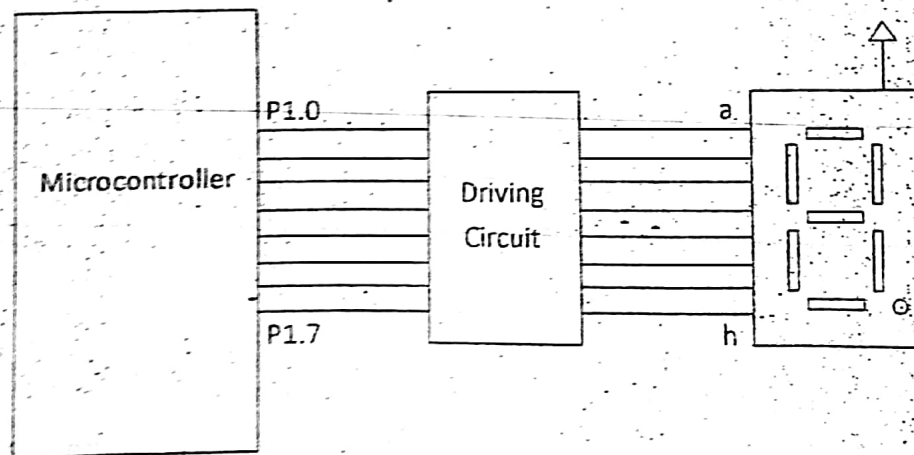


Figure 9.9: Connection of common anode seven segment with microcontroller

Coding Implementations

In Assembly language, simple MOV instructions can be used to transfer HEX value to the seven segment display. For example, MOV P2, #92 will display the digit 5 for common anode configurations. Using C programming language, simple assignment can be done. For example, P2 =

0x92; will display digit 5. However, appropriate delay or repetition mechanism must be used to ensure that the digit displays for certain duration and becomes observable to the designer or user.

Delay Calculation in assembly program

Actual time of the execution of instructions can be determined by making use of the operating frequency of the microcontroller and machine cycles required by the instructions. The total machine cycles required by the instruction is multiplied by time duration of one machine cycle to calculate the total time.

DELAY:

```

    MOV R4, #64H      ; MC = 1, executes only once
AGAN: MOV R3, #0AH    ; MC = 1, executes 100 times
AGA:  DJNZ R3, AGA     ; MC = 2, executes 100 x 10 = 1000 times
      DJNZ R4, AGAN    ; MC = 2, executes 100 times
      RET              ; MC = 2, executes 1 times

```

In 8051, crystal frequency is 11.0592MHz and one machine cycle (MC) is equal to 12 clock cycles.

So, 1 MC = 1.085µs.

Total machine cycles in DELAY subroutine = 1 + 1x100 + 2x1000 + 2x100 + 2 = 2303

Total time duration = 2303 x 1.085µs = 2.5ms

FEW SOLVED EXAMPLES

1. Write an assembly and C language program to generate a pulse of 50% duty cycle at pin P2.3 of 8051 microcontroller.

→ Problem Analysis: Duty cycle of 50% represents equal ON and OFF time at pin P2.3, so an arbitrary delay is required after setting P2.3 and after resetting P2.3.

Source Code:

```

ORG 00H
CLR P2.3
Back:
    CLR P2.3
    LCALL DELAY
    SETB P2.3
    LCALL DELAY
    SJMP Back

    ORG 300H
DELAY:
    MOV R5, #64H
AGAIN: MOV R4, #0FFH
    AGAN: MOV R3, #08H
    AGA:  DJNZ R3, AGA
        DJNZ R4, AGAN
        DJNZ R5, AGAIN
    RET

END

```

In C programming language

```

#include <at89x52.h>
void delay(unsigned char x)
{
    int i, j;
    for(j=0; j<x; j++)
        for(i=0; i<1275; i++)
            ;
}

void main()
{
    P2_3 = 0;
    while(1)
    {
        P2_3 = 1;
        delay(50);
        P2_3 = 0;
        delay(50);
    }
}

```


Control the LED connected at 2.1 by a SWITCH which is connected to P1.3. ON/OFF status of LED is defined by ON/OFF status of SWITCH.

→ Problem Analysis: This is a simple data movement problem in which a bit from one pin P1.3 of microcontroller is assigned to another pin P2.1. The code can vary based of LED configuration and SWITCH configuration used. Two cases are given below:

CASE I: The SWITCH will generate high logic when pressed and LED will glow when high logic is assigned to pin 2.1.

Source Code:

```
ORG 00H
CLR P2.1
SETB P1.3
Back:
    MOV C, P1.3
    MOV P2.1, C
    SJMP Back
END
```

```
#include<at89x52.h>
void main()
{
    P2_3 = 0;
    P1_3 = 1;
    while(1)
    {
        P2_1 = P1_3;
    }
}
```

CASE II: The SWITCH will generate low logic when pressed and LED will glow when high logic is assigned to pin 2.1.

Source Code:

```
ORG 00H
CLR P2.1
SETB P1.3
Back:
    MOV C, P1.3
    CPL C
    MOV P2.1, C
    SJMP Back
END
```

```
#include<at89x52.h>
void main()
{
    P2_3 = 0;
    P1_3 = 1;
    while(1)
    {
        P2_3 = !P1_3;
    }
}
```

3. Using an assembly and C language program, generate a pulse of 75% duty cycle at pin P1.7 when the switch connected to P1.1 is ON.

→ Problem Analysis: Duty cycle of 75% represents ON time three times more than OFF time at pin P1.7. The status of P1.1 must be checked continuously. Based on the logic level at P1.1 after button is pressed, delay in case of ON time must be three times more than that of OFF time. In the code below, we assume the switch connects P1.1 to ground when pressed and P1.1 connected to VCC when not pressed. So, a low logic at P1.1 will generate the required pulse using appropriate delay.

Source Code:

```

ORG 00H

CLR P1.7
SETB P1.1

BACK:
    MOV C, P1.1
    JG BACK
    SETB P1.7
    LCALL DELAY
    LCALL DELAY
    LCALL DELAY
    CLR P1.7
    LCALL DELAY
    SJMP BACK

ORG 300H

DELAY:
    MOV R5, #64H
AGAIN: MOV B4, #0FFH
AGAN:  MOV B3, #08H
AGA:   DJNZ R3, AGA
       DJNZ R4, AGAN
       DJNZ R5, AGAIN
       RET

```

END-

```

#include <at89x52.h>
#define OUT P1_7
#define SW P1_1

void delay(unsigned int x)
{
    int i, j;
    for(i=0; i<x; i++)
        for(j=0; j<1275; j++);
}

void main()
{
    OUT = 0;
    SW = 1;
    while(1)
    {
        if(SW == 0)
        {
            OUT = 1;
            delay(300);
            OUT = 0;
            delay(100);
        }
    }
}

```

4. Write an assembly and C language program to generate a count from 0 to 9 using a seven segment display. Use Common Cathode configurations.

→ **Problem Analysis:** The equivalent HEX values are directly assigned one by one to the required port. Certain delay after each digit display can be placed to control the speed of count.

Source Code:

```

ORG 00H
    MOV P2, #00H
BACK:
    MOV P2, #3FH
    LCALL DELAY
    MOV P2, #06H
    LCALL DELAY
    MOV P2, #5BH
    LCALL DELAY
    MOV P2, #4FH
    LCALL DELAY
    MOV P2, #66H
    LCALL DELAY
    MOV P2, #6DH
    LCALL DELAY
    MOV P2, #7DH
    LCALL DELAY
    MOV P2, #07H
    LCALL DELAY

    MOV P2, #7FH
    LCALL DELAY
    MOV P2, #6FH
    LCALL DELAY
    SJMP BACK

ORG 300H
DELAY:
    MOV R5, #64H
AGAIN:
    MOV R4, #0FFH
AGAN:
    MOV R3, #08H
AGA:
    DJNZ R3, AGA
    DJNZ R4, AGAN
    DJNZ R5, AGAIN
    RET
END
    
```

ALTERNATIVE CODE:

Problem Analysis: The HEX values are not directly assigned rather stored in memory and accessed using data pointer (DPTR). DPTR is used to point to the HEX values and MOVC instruction must be used to access the HEX values. MOVC uses accumulator to represent offset as well as data. The value of accumulator is added to DPTR to represent the address of memory and finally the data is stored into accumulator.

Source Code:

```

ORG 00H
    MOV P2, #00H
    MOV R6, #00H → counter
    MOV DPTR, #DIGITS

MAIN:
    MOV A, R6
    MOVC A, @A+DPTR
    MOV P2, A
    LCALL DELAY
    INC R6
    CJNE R6, #0AH, MAIN

    MOV R6, #00H
    SJMP MAIN

DELAY:
    MOV R3, #0F0H
DEL1: MOV R2, #0FAH
DEL2: DJNZ R2, DEL2
      DJNZ R3, DEL1
RET

DIGITS:
    DB 3FH, 06H, 5BH, 4FH, 66H
    DB 6DH, 7DH, 07H, 7FH, 6FH

```

END

```

#include <at89x52.h>
#define DISPLAY P2
void delay(unsigned int x)
{
    unsigned int i,j;
    for(i=0;i<x;i++)
        for(j=0;j<1275;j++);
}

char digits[] = {0x3F, 0x06, 0x5B, 0x4F,
0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};

void main()
{
    unsigned char i;
    DISPLAY = 0x00;
    while(1)
    {
        for(i=0;i<10;i++)
        {
            DISPLAY = digits[i];
            delay(100);
        }
    }
}

```

5. A PUSH BUTTON is connected to P1.1, increase the count in SEVEN SEGMENT when the button is pressed.

→ Source Code:

ORG 00H

SETB P1.1

MOV P2, #00H

MOV R6, #00H

MOV DPTR, #DIGITS

MAIN:

MOV A, R6

MOVC A, @A+DPTR

MOV P2, A

MOV C, P1.1

JC MAIN - (0 gnd at press)

LCALL DELAY

INC R6

CJNE R6, #0AH, MAIN

MOV R6, #00H

SJMP MAIN

DELAY: - -

MOV R3, #0FQH

DEL1: MOV R2, #0FAH

DEL2: DJNZ R2, DEL2

DJNZ R3, DEL1

RET

DIGITS:

DB 3FH, 06H, 5BH, 4FH, 66H

DB 6DH, 7DH, 07H, 7FH, 6FH

END

```
#include <at89x52.h>
#define DISPLAY P2
#define SW P1_1

void delay(unsigned int x)
{
    unsigned int i,j;
    for(i=0;i<x;i++)
        for(j=0;j<1275;j++);
}

char digits[] = {0x3F, 0x06, 0x5B, 0x4F,
0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};

void main()
{
    unsigned char i = 0;
    DISPLAY = 0x00;
    while(1)
    {
        DISPLAY = digits[i];
        if(SW == 0)
        {
            i++;
            if(i>9)
                i = 0;
            delay(20);
        }
    }
}
```

6. Using two seven segment displays, build a down counter which counts from 99 to 00. Make appropriate assumptions wherever necessary.

→ Description: Two ports can be used to display two digits. However, it is better to use a single port for both seven segment displays and control their display through control lines. Two digits separately are sent at different instant of time. Corresponding seven segment display must be enabled while the digits are sent at different instant. However, the time duration between sending of lower digit and upper digit must not be high. High delay can lead to flickering effect which causes both digits to be displayed one by one rather than simultaneously. Also the speed of count can be controlled by using appropriate repetition of each display of both digits.

Source Code:

ORG 00H

MOV P2, #00H

MOV R6, #09H

; counter for lower byte

MOV R7, #09H

; counter for upper byte

MOV R5, #07h

; to control speed of counter

MOV P3, #00H

MOV DPTR, #LABEL1

; loads the starting address of hex code list to DPTR

MAIN:

MOV A, R6

SETB P3.0

; activates 2nd display to display lower byte

CLR P3.1

; deactivates the 1st display

LCALL DISPLAY

LCALL DELAY

MOV A, R7

SETB P3.1

; activates 1st display

CLR P3.0

; deactivates the 2nd display

LCALL DISPLAY

LCALL DELAY

DJNZ R5, MAIN

; repetition of same display to control speed

MOV R5, #07H


```
DEC R6                                ; decrease value of R6
CJNE R6, #-1, MAIN                    ; compare unless R6 becomes less than zero
```

```
MOV R6, #09H
DEC R7
CJNE R7, #-1, MAIN
```

```
MOV R7, #09H
SJMP MAIN
```

DISPLAY:

```
MOVC A, @A+DPTR                      ; load HEX value to accumulator from memory
MOV P2, A                            ; sending HEX value to SEVEN SEGMENT through P2
RET
```

DELAY:

```
MOV R3, #F0H
DEL1: MOV R2, #0FAH
DEL2: DJNZ R2, DEL2
      DJNZ R3, DEL1
      RET
```

LABEL1: ; represents starting address of HEX value list

```
DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH
```

END

Equivalent code in C language

Description: Within infinite loop, the first loop is used to select the upper byte, the second loop is used to select the lower byte and the third loop is used to control the speed of count.

Source Code:

```
#include <at89x52.h>
#define DISPLAY P2
#define SELO P3.1
#define SEL1 P3.0
```

```

void delay(unsigned int x)
{
    unsigned int i,j;
    for(i=0;i<x;i++)
        for(j=0;j<1275;j++);
}

char digits[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};

void main()
{
    char i, j, k;
    DISPLAY = 0x00;
    while(1)
    {
        for(i = 9; i >= 0; i++)           // loop for upper byte
        {
            for(j = 9; j >= 0; j--)       // loop for lower byte
            {
                for(k = 0; k < 7; k++)    // loop to control the speed of count
                {
                    DISPLAY = digits[j];
                    SELO = 0;
                    SEL1 = 1;
                    delay(5);
                    DISPLAY = digits[i];
                    SELO = 1;
                    SEL1 = 0;
                    delay(5);
                }
            }
        }
    }
}

```